10

Accelerating App Development with Template Studio

Starting a new project from scratch can be daunting. What are the best practices for application architecture and project layout? **Template Studio for WinUI** is an open source project that began as **Windows Template Studio** for UWP and WPF applications. Each version of Template Studio is an extension for Visual Studio. It now supports WinUI, and variations have been created that will also generate **.NET MAUI** and **Uno Platform** projects.

In this chapter, we will cover the following topics:

- Discover what Template Studio is and how it can help developers create new WinUI projects while following best patterns and practices
- Review the history of Windows Template Studio and UWP applications
- Create a new project with Template Studio for WinUI and the MVVM Toolkit
- Learn about other variations of Template Studio

By the end of this chapter, you will understand the origins of the current Template Studio extensions for Visual Studio. You'll also have enough familiarity with Template Studio for WinUI to choose it when starting your next WinUI 3 project. You will also know where you can

go to suggest enhancements, submit issues, or improve the project by submitting your own changes to the open source project.

Technical requirements

To follow along with the examples in this chapter, please refer to the *Technical requirements* section of ***Chapter 2***, *Configuring the Development Environment and Creating the Project*.

The source code for this chapter is available on GitHub here: **https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/master/Chapter10**

Overview of Template Studio for WinUI

Template Studio for WinUI is an open source extension for Visual Studio that enhances the experience of creating a new WinUI 3 project. Microsoft currently has three versions of the Template Studio extension available:

- **Template Studio for WinUI (C#)** – This is the extension we will use in this chapter. There are also plans for a C++ version of Template Studio for WinUI.
- **Template Studio for WPF** – The WPF version of Template Studio is similar to the WinUI extension. It creates a .NET WPF project.
- **Template Studio for UWP** – This version is the updated extension that was originally named Windows Template Studio.

The code and documentation for the Template Studio projects are available on GitHub: **https://github.com/microsoft/TemplateStudio**. The team also maintains a release roadmap on GitHub that you can monitor:

https://github.com/microsoft/TemplateStudio/blob/main/docs/roadmap.md.

Creating a new project with one of the Template Studio extensions launches an enhanced wizard-style experience where you can select the type of project you want to create and the design pattern to follow (such as MVVM). You can also select some pre-defined pages and other features to include in your project, as well as a unit test project. We will step through the full experience and discuss the options in the next section.

Template Studio for WinUI is installed like any other Visual Studio extension. If you aren't familiar with the process, you can either download the **VSIX** package from the Visual Studio Marketplace (https://marketplace.visualstudio.com/items?itemName=TemplateStudio.TemplateStudioForWinUICs) or you can search for and add it in the **Manage Extensions** dialog box in Visual Studio.

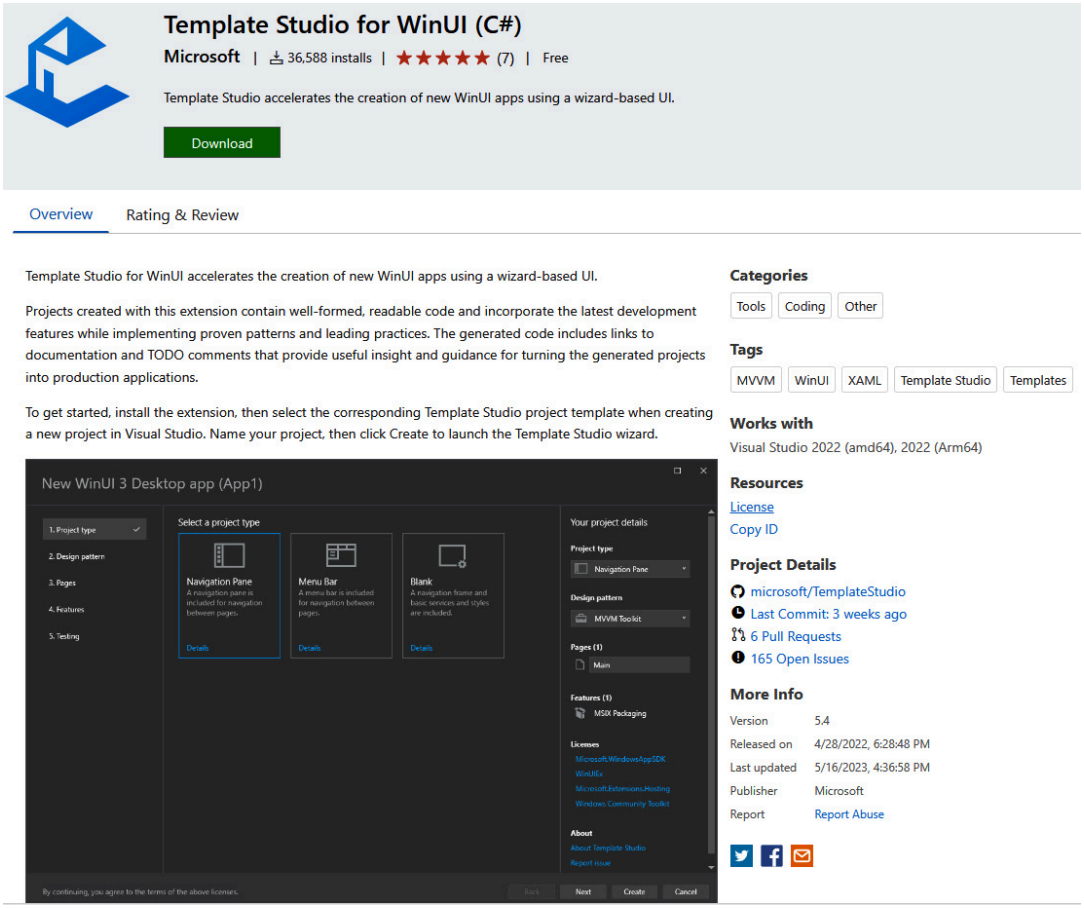

Figure 10.1 – Template Studio for WinUI in Visual Studio Marketplace

We'll be using the **Manage Extensions** dialog to install the extension in this chapter. For detailed information on managing extensions in Visual Studio, see this documentation on Microsoft Learn: **https://learn.microsoft.com/visualstudio/ide/finding-and-using-visual-studio-extensions**.

NOTE

If you're curious about the inner workings of a Visual Studio extension package, or VSIX, you can read more about them on Microsoft Learn: **https://learn.microsoft.com/visualstudio/extensibility/anatomy-of-a-vsix-package**

The code that is generated by the extension continues to guide you as you proceed to build out the newly created project. There are *TODO* comments with guidance about where you add your own code and helpful links to documentation that explains the concepts and controls used in the code. The extension is updated frequently to reflect the latest WinUI features, practices, and recommendations for Windows developers.

Contributions to the Template Studio projects on GitHub are all reviewed to ensure they follow good coding style, fluent design, and helpful comments throughout.

Now that we have reviewed some basics of Template Studio for WinUI, let's dive right in and create a new project with the extension.

Starting a new WinUI project with Template Studio

In this section, we are going to create a new WinUI project with Template Studio for WinUI. We'll install the extension, create the project with several pages and features selected, and then run the project to explore what is provided before we add any of our own code. In the next section, we'll dive a little deeper into the generated code to see where we would start extending and enhancing the project if we were building a production application:

1. To start, open Visual Studio and select **Continue without code** to open the IDE without loading a solution:

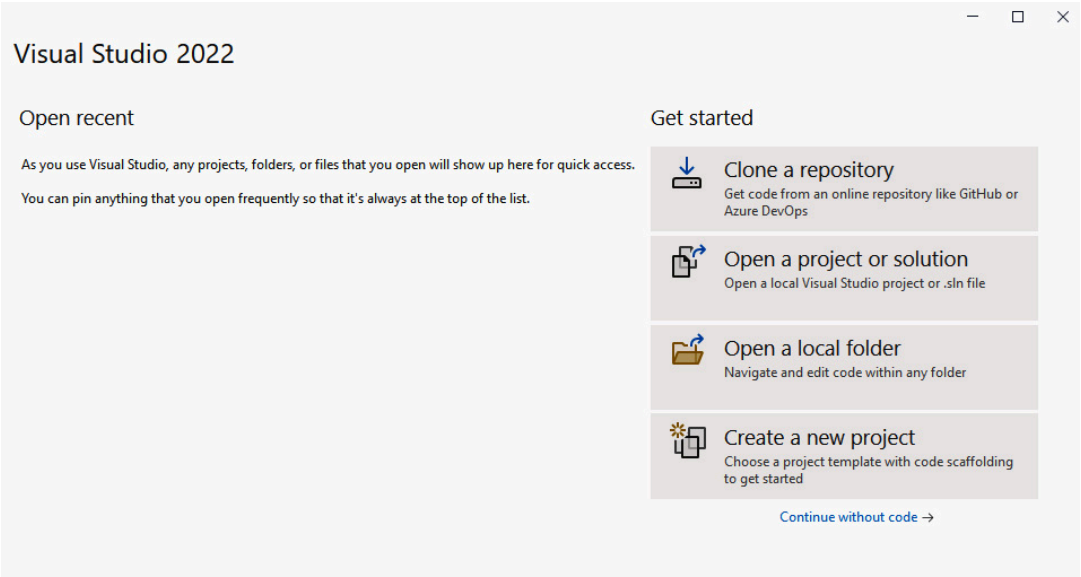

Figure 10.2 – The Visual Studio 2022 launch dialog

2. In the menu, select **Extensions** | **Manage Extensions** to open the **Manage Extensions** dialog:

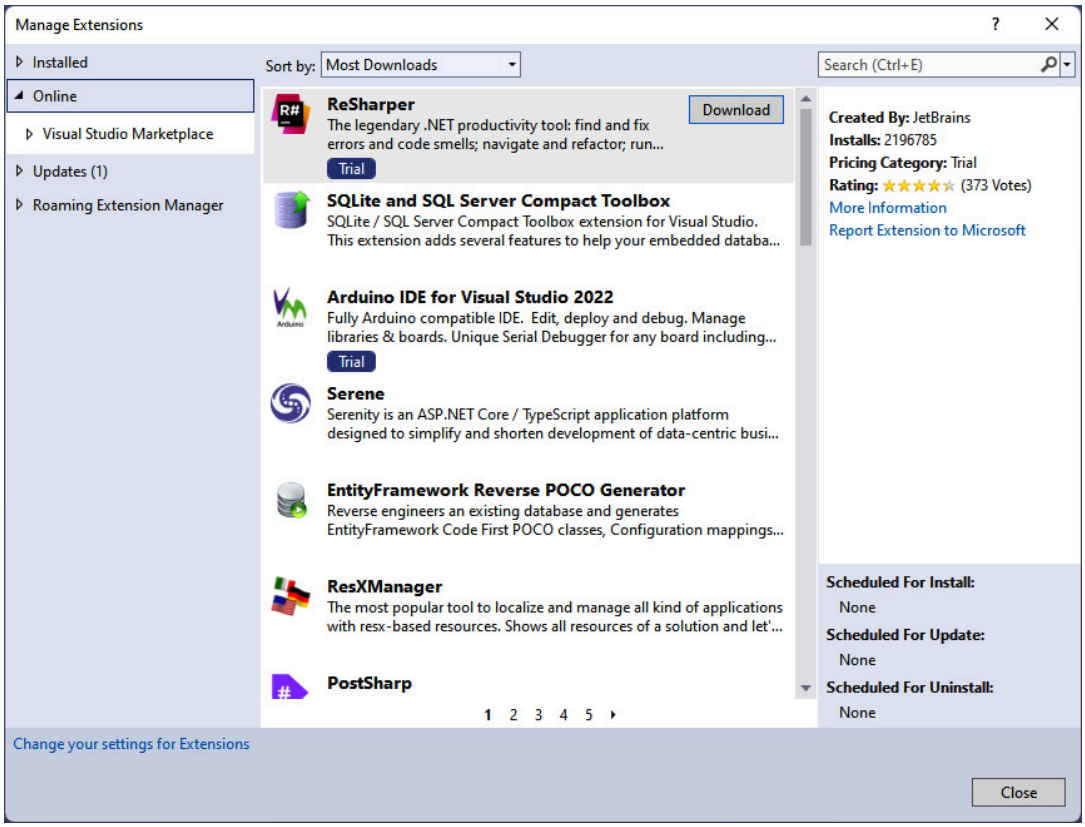

Figure 10.3 – The Manage Extensions dialog in Visual Studio

3. In the **Search** field, search for **`template studio`**. In the list of search results, select **Download** on **Template Studio for WinUI (C#)**:

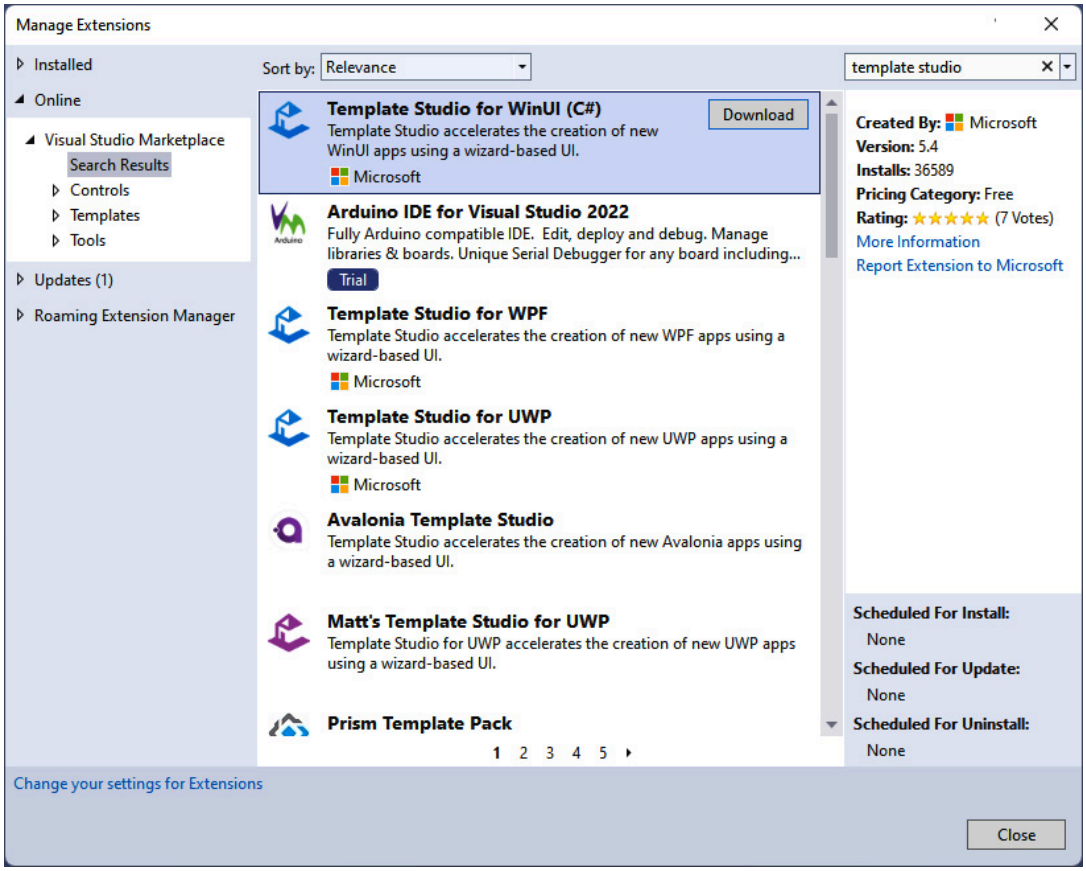

Figure 10.4 – Installing Template Studio for WinUI from Manage Extensions

You will need to restart Visual Studio to complete the extension's installation.

4. After the installation completes, launch Visual Studio and select **Create a new project**.
5. On the **Create a new project** screen, if you don't see **Template Studio for WinUI** in the list of templates, search for **`template studio`**. The new project template should appear first in the list of results:

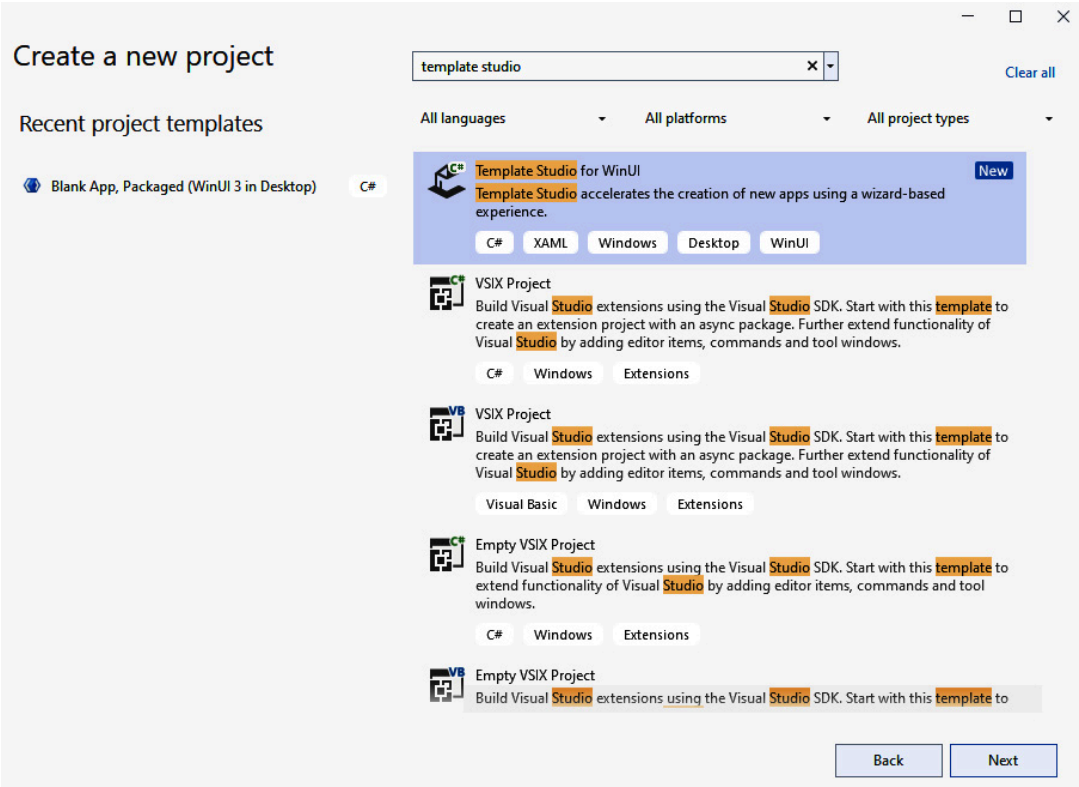

Figure 10.5 – Selecting Template Studio for WinUI to create a new project

6. Select **Next** and, on the **Configure your new project** screen, give the project a name such as **`TemplateStudioSampleApp`**.
7. Click **Create** to continue. Instead of the Visual Studio IDE immediately loading, you will be presented with the Template Studio wizard:

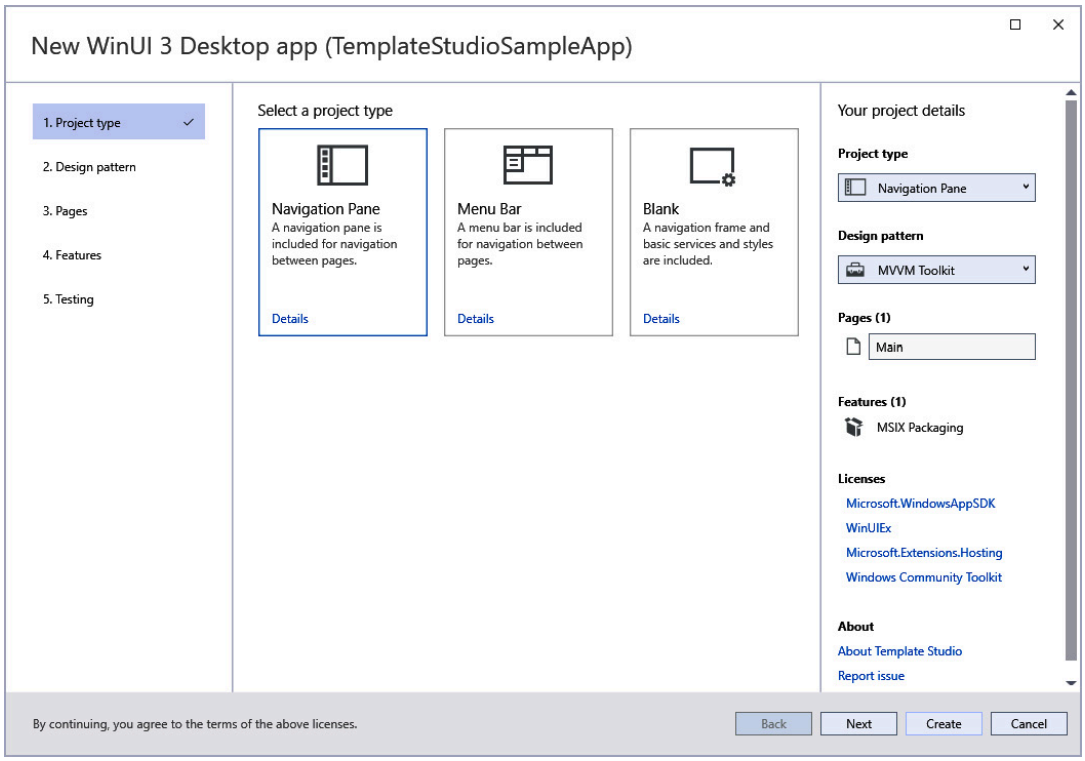

Figure 10.6 – The first page of the Template Studio for WinUI wizard

The first page of the wizard prompts you to select a project type. The choices are **Navigation Pane**, **Menu Bar**, or **Blank**. You can select **Details** on any of the choices to read more about each option. Select the **Navigation Pane** option. This layout leverages the **`NavigationView`** control that we discussed in ***Chapter 5***, *Exploring WinUI Controls*. It's also the navigation method used in the WinUI 3 Gallery app.

8. Select **Next** to continue. On the **Design pattern** screen, the only available option is **MVVM Toolkit**. Some versions of Template Studio offer other MVVM frameworks or a **Code Behind** option. For WinUI 3, we only have one choice:

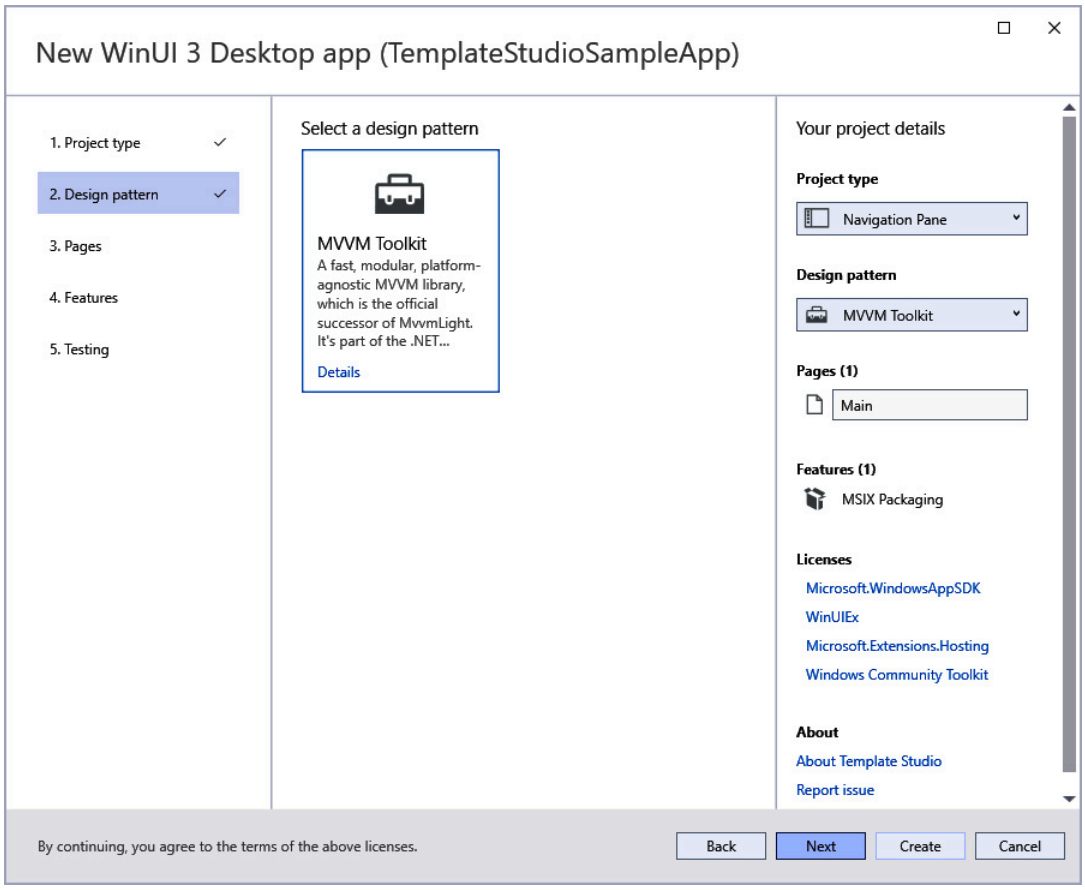

Figure 10.7 – The Design pattern screen

You'll also notice that the right pane contains the **Your project details** heading and the options that have been selected so far. When you're more familiar with Template Studio, you will be able to review these options and click **Create** to finish the wizard without visiting each screen if you're satisfied with the default selections.

9. Click **Next** and review the **Pages** screen:

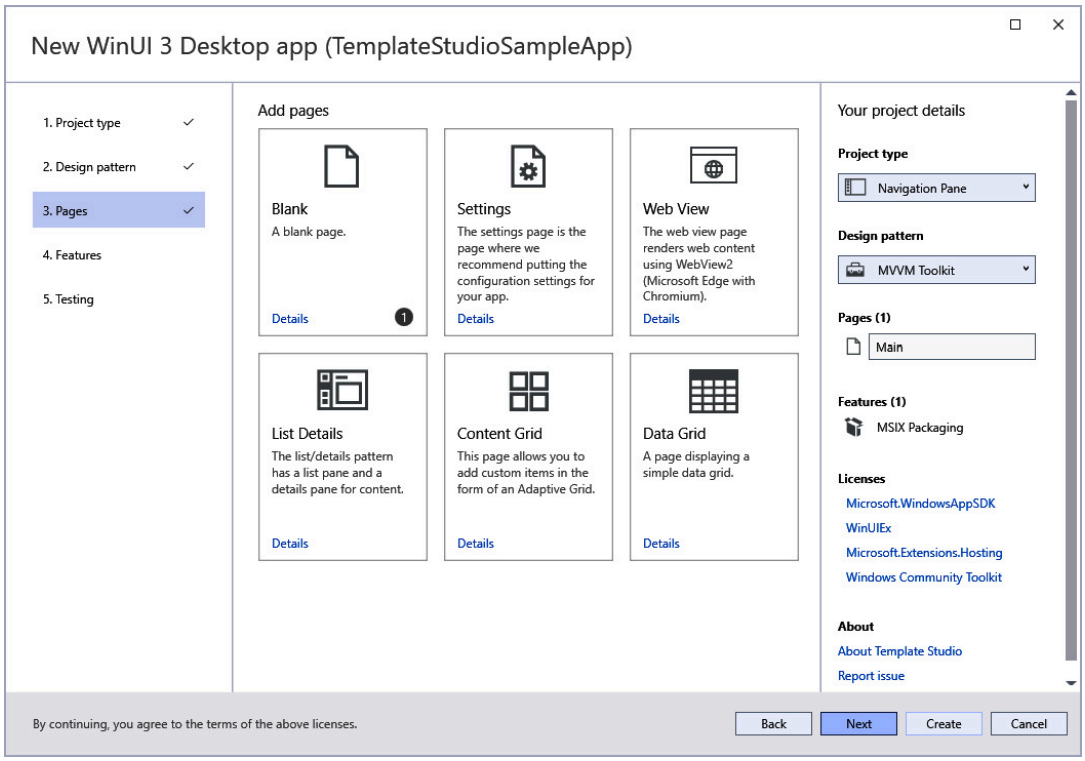

Figure 10.8 – The Pages screen in Template Studio for WinUI

By default, one **Blank** page has been selected. The default name of this page is **Main**. You can change the names of any pages in the **Your project details** panel.

10. Let's select several other page types to review in the next section. Select **Data Grid**, **List Details**, **Web View**, and **Settings**. You can select more if you like:

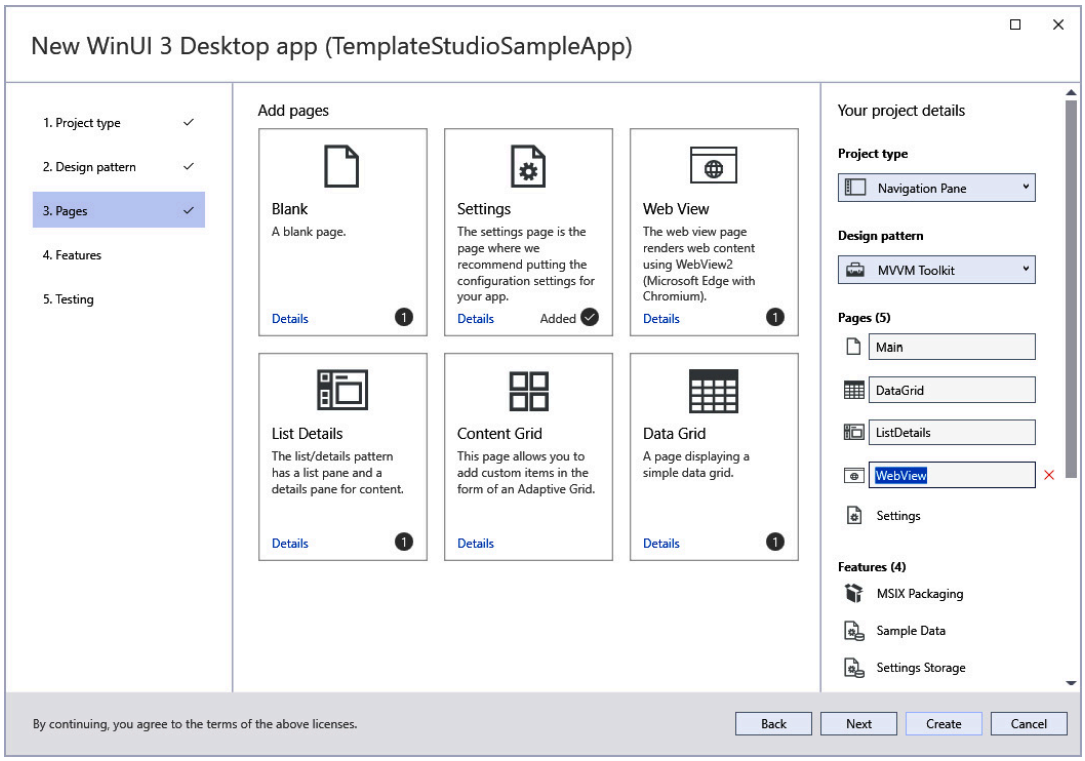

Figure 10.9 – Selecting several additional pages for our project

Any of the pages can be renamed except for **Settings**. We'll leave the default names.

11. Click **Next** to move on to the **Features** screen:

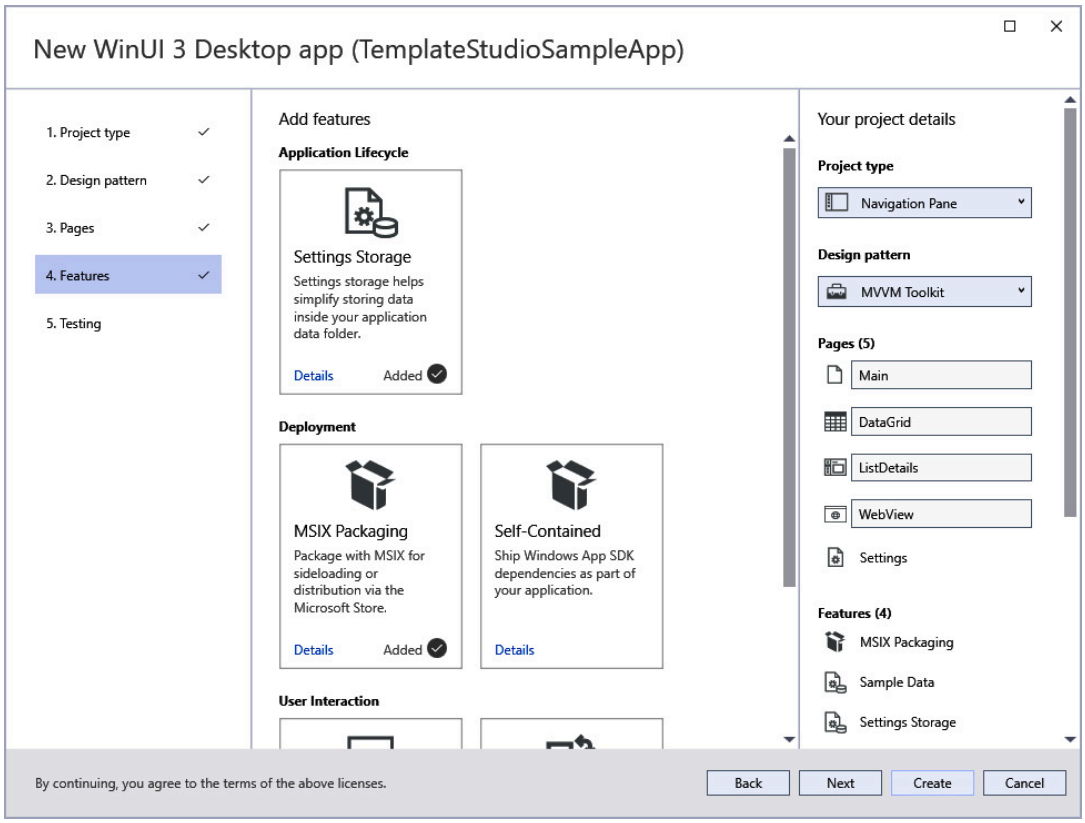

Figure 10.10 – Exploring the Features screen of Template Studio for WinUI

The default selections on the **Features** screen are **Settings Storage**, **MSIX Packaging**, and **Theme Selection**. We'll leave those defaults selected. If you want to include the Windows App SDK dependencies with your project, you can select **Self-Contained**. This is useful if you are going to distribute your app manually with an **xcopy deployment**. We'll discuss deployment options in ***Chapter 14***, *Packaging and Deploying WinUI Applications*. The other unselected option is **App Notifications**. We have already explored notifications in ***Chapter 8***, *Adding Windows Notifications to WinUI Applications*. If you want to learn more about any of the features, select **Details**.

12. Click **Next** to move on to the final step of the wizard, **Testing**:

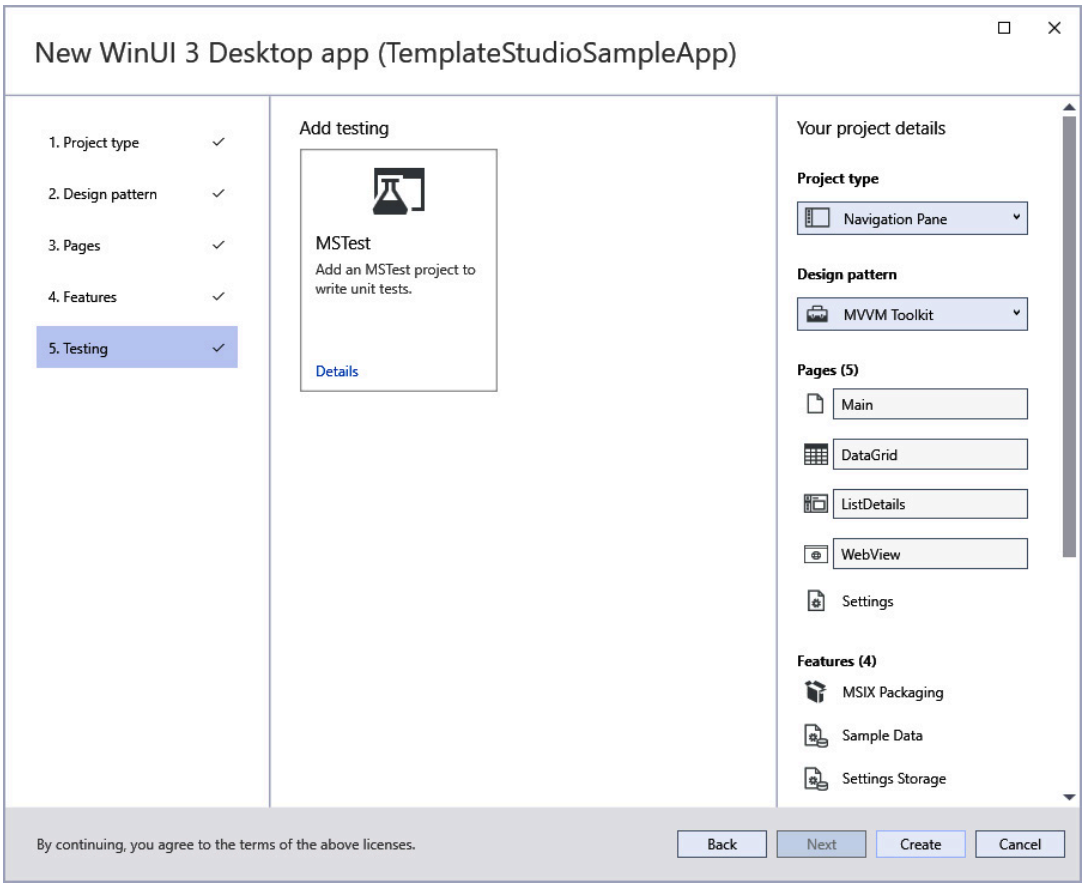

Figure 10.11 – The Testing screen of the Template Studio for WinUI wizard

13. The only option on the **Testing** screen is **MSTest**. Select it to add a unit test project to your solution.
14. We've reached the end of the wizard. Select **Create** to create the solution and launch the Visual Studio IDE:

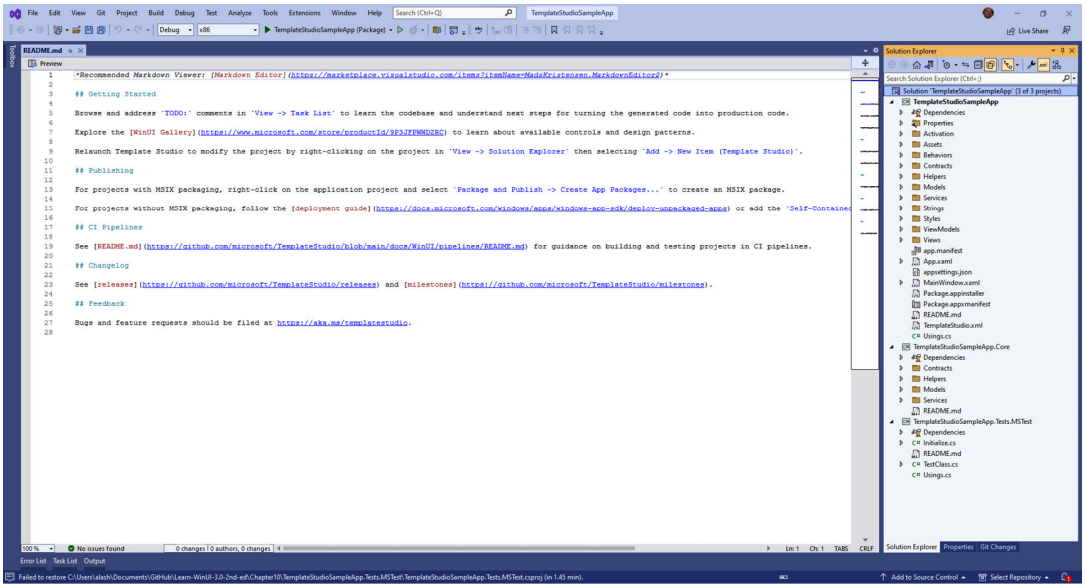

Figure 10.12 – Visual Studio with our new solution loaded

15. The markdown file named **`README.md`** will be opened by default. If you want to view a formatted preview of the markdown, you can select the **Preview** button at the top of the editor window:

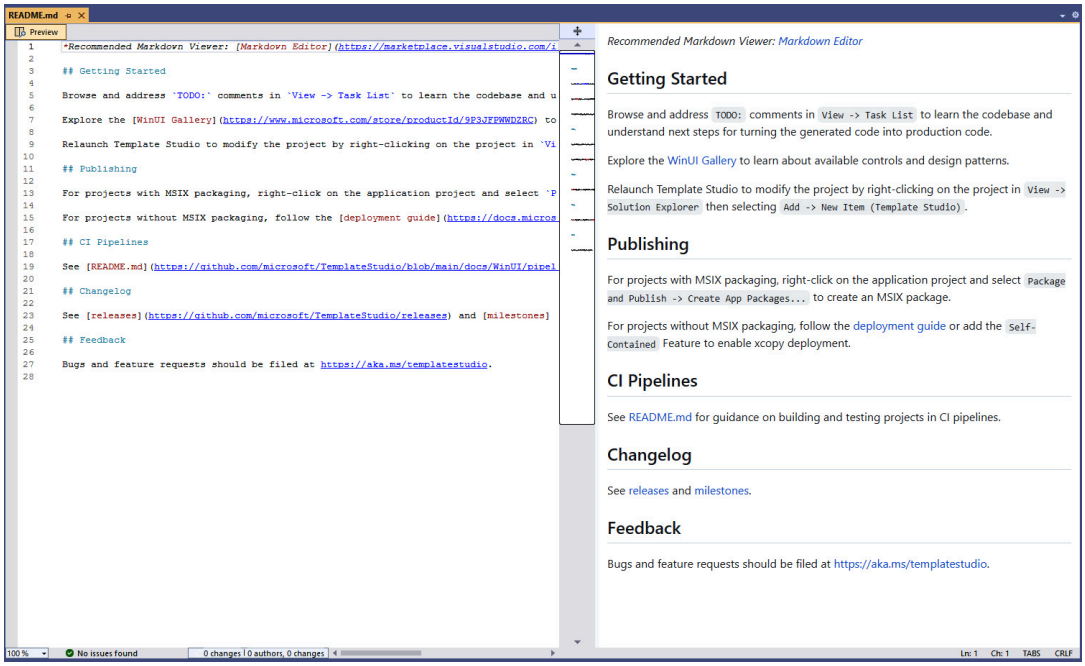

Figure 10.13 – Viewing the formatted README file

The solution contains three projects: **TemplateStudioSampleApp**, **TemplateStudioSampleApp.Core**, and **TemplateStudioSampleApp.Tests.MSTest**. We'll discuss the purpose and contents of each of these projects in the next section:

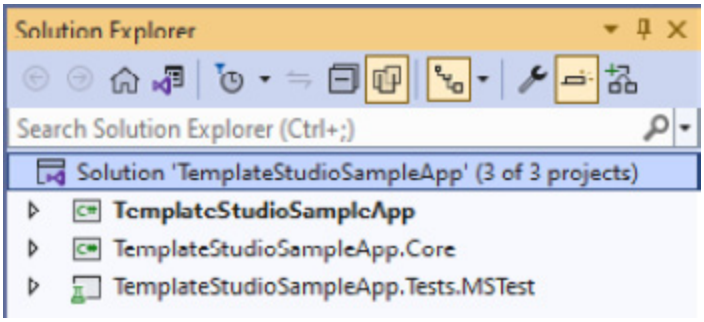

Figure 10.14 – Viewing the projects in Solution Explorer

16. It's finally time to run the application. Start debugging and ensure that there are no compile-time or runtime errors when launching

it:

Figure 10.15 – Running the TemplateStudioSampleApp solution

Everything should run as expected, and you can try navigating to each page with the **`NavigationView`** controls on the left. The **Settings** button will always appear at the bottom of the navigation bar.

Now that we have a working application, let's take some time to understand what's been generated for us.

Exploring the code generated by Template Studio

It's time to review the contents of the newly generated **TemplateStudioSampleApp** solution. There is a lot of code created for us in the three projects, so we won't be exploring every file. We'll discuss each folder, explore a few of the important files, and make some minor code changes along the way. Each project has its own **`README.md`** file with an overview of the project and its purpose.

Exploring the Core project

The **TemplateStudioSampleApp.Core** project is a class library project targeting .NET 7. This is where you would place any code that is in-

tended to be shared across projects. The project contains four folders:

- **`Contracts`**: Interfaces for the services in the project are kept here in a **`Services`** subfolder. Any new interfaces needed in this project should be created here.
- **`Helpers`**: This folder contains a **`Json`** helper class with methods to convert between **JSON** strings and .NET objects. Add your own common helper classes to this folder.
- **`Models`**: There are some sample model classes in this folder that are used to populate company and order data on the **`DataGridPage`** and **`ListDetailPage`** views.
- **`Services`**: This folder contains **`FileService`** and **`SampleDataService`** classes. The **`FileService`** class reads and writes files with classes in the .NET **`System.IO`** and **`System.Text`** namespaces. The **`SampleDataService`** creates static data to populate the model classes for the UI. Any other shared services can be added here.

NOTE

It's possible that the Template Studio extensions may be updated to create .NET 8 libraries by the time this book is published.

Next, let's explore the largest project in the solution, **TemplateStudioSampleApp**.

Exploring the main project

The main project, **TemplateStudioSampleApp**, is the largest of the three projects in the solution. It contains all the user interface markup and logic. You will probably recognize the files in the root of the project, after working with other WinUI projects. **App.xaml**, **MainWindow.xaml**, and **Package.appxmanifest** are all there, along with C# code-behind files for each XAML file.

The contents of the **Views** and **ViewModels** folders also contain files that you would expect to find there. We have a corresponding **`Page`** and **`ViewModel`** class for each page that we selected when configuring the project with the Template Studio wizard. There are also **`ShellPage`** and **`ShellViewModel`** classes for handling the navigation between child pages. **ShellPage.xaml** contains the **`NavigationView`** control and is the direct child of **`MainWindow`**.

Next, let's go through the contents of the remaining folders:

- **`Activation`**: This folder contains the **`ActivationHandler`**, **`DefaultActivation Handler`**, and **`IActivationHandler`** classes. They are helpers that use the **`INavigationService`** to navigate to the selected page inside the **`Frame`** hosted within **`ShellPage`**. If you needed to change the default activation behavior, you would create a new handler that inherits from **`ActivationHandler`**.
- **`Assets`**: This contains the graphical assets for the project. The **`Assets`** folder is present in every WinUI project.
- **`Behaviors`**: This folder contains an enum named **`NavigationViewHeaderMode`**, which specifies the appearance of the navigation bar, and **`NavigationViewHeaderBehavior`**, which contains the logic to implement these modes. The behavior class inherits from **`Behavior<NavigationView>`**. Any other custom WinUI behaviors would be added here.
- **`Contracts`**: This is where you keep your interfaces for the project. The two subfolders, **`Services`** and **`ViewModels`**, contain interfaces for eight existing classes in those corresponding project folders.
- **`Helpers`**: Any helper classes specific to this project are kept here. Some of the helpers provided by default include **`NavigationHelper`**, **`SettingsStorageExtensions`**, and **`TitleBarHelper`**. The **`SettingsStorageExtensions`** helper class leverages members of the **`Windows.Storage`** and **`Windows.Storage.Streams`** namespaces to read and write user settings.

- **`Models`**: The sample data models are kept in the **Core** project, but there's also one model class in this project. The **`LocalSettingsOptions`** class stores the name and location of the settings file. Any other UI-specific model classes would be added here.
- **`Services`**: There are seven service classes in the **`Services`** folder. You can probably determine the purpose of each by its name: **`ActivationService`**, **`LocalSettings Service`**, **`NavigationService`**, **`NavigationViewService`**, **`PageService`**, **`ThemeSelectorService`**, and **`WebViewService`**. You should take some time to review the code in these services.
- **`Strings`**: This folder holds language-specific **`Resources.resw`** files to localize any string values displayed in your application. The English language resource file can be found in the **`en-us`** folder. For more information about localization, you should read *Localize your WinUI 3 app* on Microsoft Learn: **https://learn.microsoft.com/windows/apps/winui/winui3/localize-winui3-app**.
- **`Styles`**: This folder contains three files with XAML **`ResourceDictionary`** elements: **`FontSizes.xaml`**, **`TextBlock.xaml`**, and **`Thickness.xaml`**. Let's look at the contents of **`FontSizes.xaml`** as an example:

```
<ResourceDictionary
    xmlns="http://schemas.microsoft.com/winfx/
      2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/
      2006/xaml">
    <x:Double x:Key="LargeFontSize">24</x:Double>
    <x:Double x:Key="MediumFontSize">16</x:Double>
</ResourceDictionary>
```

By centralizing how your app defines a large or medium font size, you can change this later without having to modify multiple XAML views. If you are using a **`TextBlock`** or **`RichTextBlock`**, the other option for font sizing is to use the **XAML type ramp** (discussed in

Chapter 7, *Fluent Design System for Windows Applications*) defined in the XAML theme resources: **https://learn.microsoft.com/windows/apps/design/style/xaml-theme-resources#the-xaml-type-ramp**.

Before we move on to explore the **MSTest** project, let's look inside the **`App.xaml.cs`** file. Everything in here should look familiar to you. The project is using the **`IHostBuilder.ConfigureServices`** method to add the **`ActivationHandler`** and all services, views, and ViewModel classes to the DI container. The services and core services are added as singleton objects while all the other classes are added with **`AddTransient`**.

The **`App`** class also has a handy helper method to fetch an instance from the DI container:

```
public static T GetService<T>() T : class
{
    if ((App.Current as App)!.Host.Services.GetService
      (typeof(T)) is not T service)
    {
        throw new ArgumentException($"{typeof(T)} needs to
          be registered in ConfigureServices within
            App.xaml.cs.");
    }
    return service;
}
```

This encapsulates some exception-handling logic in case the requested type cannot be found, reducing the need to repeat this code throughout the application.

Notice how each class has helpful comments with links to Microsoft Learn resources for any related APIs or other topics. The **`App`** class has links to .NET dependency injection, logging, and other learning resources.

Let's finish this section with a quick look at the **MSTest** project that was created by Template Studio.

Exploring the MSTest project

The **TemplateStudioSampleApp.Tests.MSTest** project contains a single test class named **`TestClass`**, a **`README.md`** file, and **`Initialize`** and **`Usings`** classes. You should read the **`README.md`** file carefully. It contains information about the project, testing UI elements, dependency injection, and mocking. There are examples of how to leverage DI and mocks to test the **`SettingsViewModel`** without actually reading or writing any settings to the filesystem.

The **`TestClass`** contains examples of test initialization and cleanup at the class and test level, as well as two sample test methods: **`TestMethod`** and **`UITestMethod`**:

```
[TestMethod]
public void TestMethod()
{
    Assert.IsTrue(true);
}
[UITestMethod]
public void UITestMethod()
{
    Assert.AreEqual(0, new Grid().ActualWidth);
}
```

Any test method decorated with the **`UITestMethod`** attribute will run on the UI thread. The generated code does not test any members of the **TemplateStudioSampleApp** or **TemplateStudioSampleApp.Core** projects yet. It's up to you to add your own test methods to do that. You can get started by reading the Visual Studio unit testing documentation on Microsoft Learn:

https://learn.microsoft.com/visualstudio/test/getting-started-with-unit-testing.

In the next section, we will discuss some other Template Studio extensions created by Microsoft and one from a third party.

Template Studio extensions for other UI frameworks

We've taken a deep dive into the Template Studio for WinUI extension, but what about the UI frameworks? As we mentioned earlier in the chapter, Microsoft also maintains extensions for Template Studio for WPF and Template Studio for UWP. In fact, they even have a **Microsoft Web Template Studio** available: **https://github.com/microsoft/WebTemplateStudio**. It supports **React**, **Angular**, or **Vue.js** for the frontend and **Node.js**, **Flask**, **Molecular**, and **ASP.NET Core** as backend frameworks. If you're interested in web development, you should check it out.

There is also an extension called **MAUI App Accelerator** (**https://marketplace.visualstudio.com/items?itemName=MattLaceyLtd.MauiAppAccelerator**), which is a Template Studio version for .NET MAUI. We'll remain focused on WinUI and WPF templates in this chapter. The final two we'll review are Template Studio for WPF and the Template Studio-style wizard that comes with the Uno Platform extension for Visual Studio. Let's start with WPF.

Template Studio for WPF

The Template Studio for WPF extension (**https://marketplace.visualstudio.com/items?itemName=TemplateStudio.TemplateStudioForWPF**) is similar to its WinUI counterpart. It has one additional step in the wizard

(**Services**), and there are a few different options on some of the pages. One of the project types available to WPF developers is the **Ribbon** type. This creates a shell with a Microsoft Office-style ribbon control at the top in place of the standard menu control you would get in a **MenuBar**-type project.

The **Design pattern** screen allows you to select **Code Behind** or **Prism**, in addition to the **MVVM Toolkit** option. While the **Page** options are the same as WinUI, the **Features** screen has an option with the ability to show multiple views in separate windows. The **Services** screen has two identity-related options: **Forced Login** and **Optional Login**. Finally, the **Testing** screen has seven different options, rather than just **MSTest**. The additional testing frameworks provided are **NUnit**, **xUnit**, and **Appium** (for UI tests), and there are options to add test projects for the main project and the **Core** library project.

The generated projects have the same folder structure as the WinUI projects. Here's an example of a generated WPF project with an empty ribbon control:

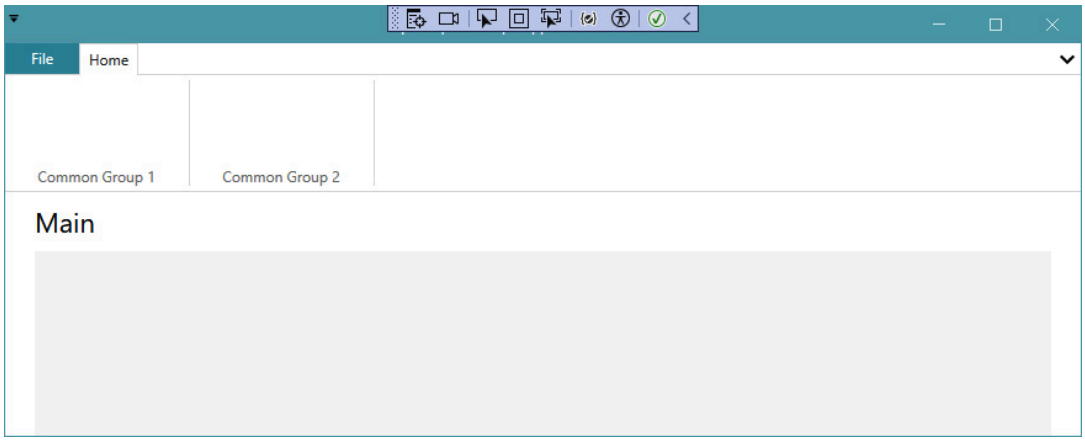

Figure 10.16 – A WPF project created by Template Studio

Let's finish up by taking a look at the Template Studio wizard provided by the Uno Platform extension.

Template Studio for Uno Platform

Uno Platform (**https://platform.uno/**) is a UI framework that uses WinUI XAML and .NET code, and it can create applications to target virtually every available device platform today: Windows, iOS, Android, macOS, Tizen, web (with **WebAssembly**), and even Linux. The Uno Platform extension for Visual Studio (**https://marketplace.visualstudio.com/items?itemName=unoplatform.uno-platform-addin-2022**) includes a new project wizard that is based on the Template Studio code.

If you install the extension in Visual Studio and create a new project with the **Uno Platform App** template, clicking the **Create** button from the **Configure your new project** screen will launch the wizard:

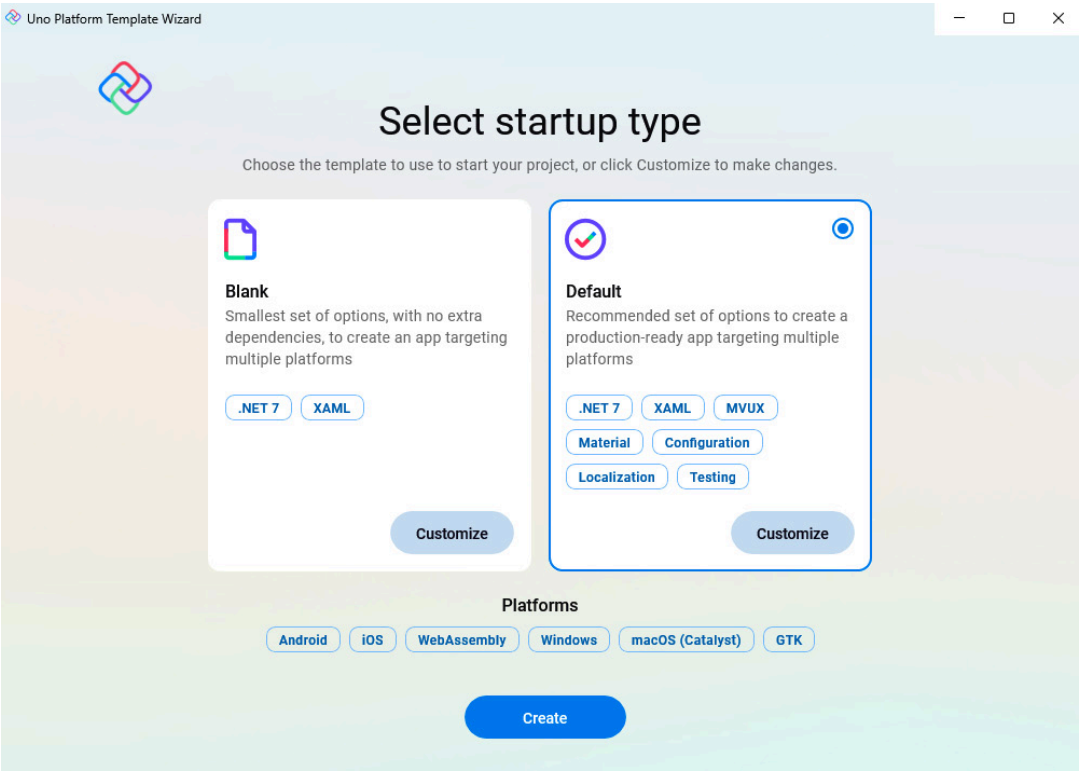

Figure 10.17 – The Uno Platform new project screen

From here, you can simply select **Create** to accept all the defaults and generate a new solution. However, if you select the **Customize** button on the **Default** startup type, you will get the full wizard experience:

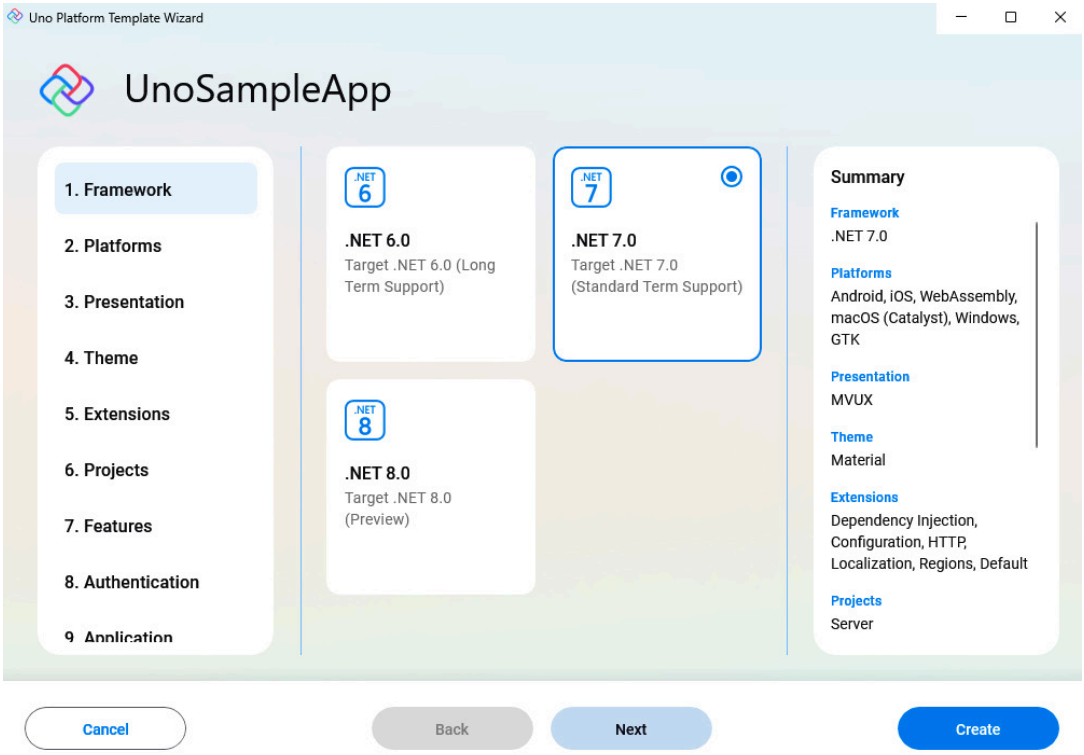

Figure 10.18 – The Uno Platform new project wizard

From here, you can select from options in 10 different categories. Even though the wizard is styled a bit differently, you can see that it has the same origins as the Template Studio extensions. We will get into the details of each step of the wizard when we build an Uno Platform app in ***Chapter 13***, *Take Your App Cross-Platform with Uno Platform*. If you create a new Uno Platform project with the defaults and run the **UnoApp1.Wasm** project, you'll see your app running in the browser by leveraging WebAssembly:

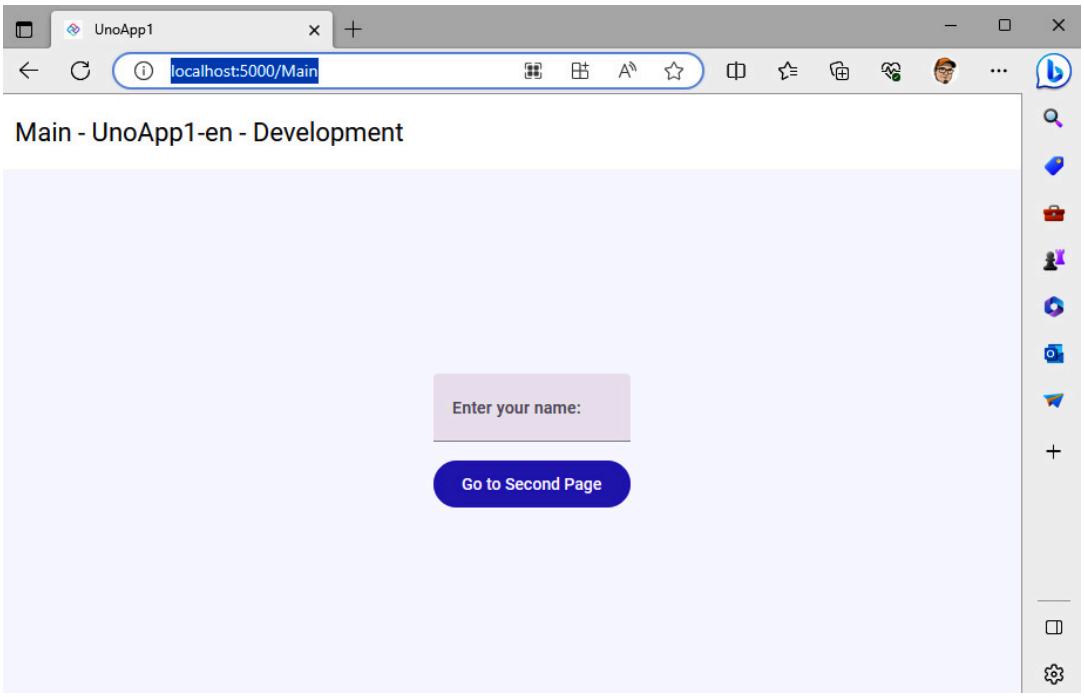

Figure 10.19 – Running an Uno Platform app in the browser

That's very cool! Now let's wrap up and review what we've learned in this chapter.

Summary

In this chapter, we learned how the Template Studio for WinUI extension can save time and promote good patterns and practices when starting a new WinUI project. We stepped through the creation of a new WinUI project with the wizard and explored the generated code in the new solution. Understanding the structure and purpose of the solution's components will make it easier to extend it for your own projects. We wrapped things up by discussing the Template Studio extensions that are available for other UI frameworks such as WPF and Uno Platform.

In the next chapter, ***Chapter 11***, *Debugging WinUI Apps with Visual Studio*, we will explore the tools and options provided by Visual Studio to make .NET and XAML developers' lives easier.

Questions

1. Which unit test framework is used when creating a test project in Template Studio for WinUI?
2. Which project type in Template Studio for WinUI implements a **`NavigationView`** control?
3. What was the previous name of Template Studio for UWP?
4. Which MVVM framework is used by Template Studio for WinUI when generating a project?
5. What file format is used for Visual Studio extensions?
6. Which Template Studio extension could you use to create a new project if you needed to include Linux as one of your target platforms?
7. Which folder contains all the interfaces for the main project generated by Template Studio for WinUI?